Here are some column definitions using the migration types and options:

```
add_column :orders, :attn, :string, limit: 100
add_column :orders, :order_type,  :integer
add_column :orders, :ship_class, :string, null: false, default: 'priority'
add_column :orders, :amount, :decimal, precision: 8, scale: 2
```

Renaming Columns

When we refactor our code, we often change our variable names to make them more meaningful. Rails migrations allow us to do this to database column names too. For example, a week after we first added it, we might decide that `e_mail` isn't the best name for the new column. We can create a migration to rename it using the `rename_column` method:

```
class RenameEmailColumn < ActiveRecord::Migration
  def change
    rename_column :orders, :e_mail, :customer_email
  end
end
```

As `rename_column` is reversible, separate `up` and `down` methods aren't required in order to use it.

Note that the rename doesn't destroy any existing data associated with the column. Also be aware that renaming isn't supported by all the adapters.

Changing Columns

`change_column` Use the `change_column` method to change the type of a column or to alter the options associated with a column. Use it the same way you'd use `add_column`, but specify the name of an existing column. Let's say that the order type column is currently an integer, but we need to change it to be a string. We want to keep the existing data, so an order type of `123` will become the string `"123"`. Later, we'll use noninteger values such as `"new"` and `"existing"`.